

YAŞAR UNIVERSITY
DEPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING

EEE 3134 MICROCONTROLLER PROJECT REPORT

SPEED RADAR

Arda ÖZTÜRK

ELECTRICAL AND ELECTRONICS ENGINEERING

PRESENTATION DATE: 06/05/2026

BORNOVA / İZMİR

Table of Contents

CHAPTER I	2
1.1	2
CHAPTER II	2
2.1	3
CHAPTER III	2
3.1	4
CHAPTER IV	2
4.1	7
CHAPTER V	2
5.1	11
5.2	Hata! Yer işareti tanımlanmamış.
5.3	Hata! Yer işareti tanımlanmamış.
REFERENCES	3
APPENDIX-CODE	4

CHAPTER I

1.1 INTRODUCTION

Engineering Problem and Technical Description

The main engineering problem we tried to solve in this project is measuring the speed of a moving object accurately by using only low-cost components and a microcontroller. Even though calculating speed is a basic physics formula ($V = d/t$), doing it in real-time with electronic sensors is harder than it sounds. The system has to detect exactly when an object passes two fixed points, measure the time difference between these two events with millisecond accuracy and then convert this time data into a speed value that the user can read on a display. All of this must happen fast enough that the result is shown almost immediately after the object passes.

In our system, we used two infrared sensors placed at a known distance from each other. When an object passes in front of the first sensor, the microcontroller records the timestamp using its internal `millis()` function. When the same object reaches the second sensor, a second timestamp is taken. The difference between these two values gives us the travel time, and since the distance between sensors is already known, the speed can be calculated using the formula $V = d/\Delta t$. The result is then displayed on an OLED screen and the user is also warned by a buzzer and an RGB LED if the speed goes above a defined limit.

Aim of the Project

The aim of this project is to design and build a fully working microcontroller-based speed radar system. We did not just want to show a measurement on the screen, our goal was to make a small but complete device that has multiple operating modes, persistent settings, statistical analysis of past measurements, and a user-friendly menu system controlled by a joystick. The device should be able to function on its own without needing a computer connection, which means all settings must be stored in non-volatile memory and the user should be able to change them through the on-board interface.

Real-world Applications

The methodology we developed in this lab-scale project is actually used in many real-world systems. Some examples are:

- Traffic Management: Automated speed cameras used by traffic police to detect over-speeding vehicles. Our system uses the same time-of-flight principle, only on a smaller scale.
- Industrial Automation: In factories, the speed of products on conveyor belts is constantly measured to control production rate and detect malfunctions. The same two-sensor approach is commonly used here.
- Logistics and Robotics: Automated guided vehicles (AGVs) in modern

warehouses need to know their own speed to avoid collisions. Radar-like sensor systems are used for this purpose.

- Sports timing: Devices that measure the speed of a ball, a runner, or a vehicle in sports use the same principle of two timing gates.

In all these examples, the basic engineering challenge is the same as ours: measure time precisely between two events, and convert it to speed using a known distance.

CHAPTER II

2.1 MICROCONTROLLER

General Information and Applications

A microcontroller is basically a small computer placed on a single chip. Different from a normal computer, a microcontroller has its CPU, RAM, flash memory and input/output pins all integrated together, which makes it ideal for embedded systems where size, power and cost are limited. Microcontrollers are everywhere in daily life from microwave ovens, washing machines and TV remotes to car engine control units and industrial robots. Anywhere there is a small electronic device that interacts with the physical world through sensors or actuators, there is most likely a microcontroller behind it.

In this project, we used the STMicroelectronics STM32F446RE, which is mounted on a Nucleo-64 development board. This is a 32-bit ARM Cortex-M4 based microcontroller running at 180 MHz, with 512 KB flash memory and 128 KB SRAM. The Nucleo board makes it easy to use because it has Arduino-compatible headers, an integrated ST-Link programmer/debugger, and we can program it directly through USB without needing a separate programmer.

Why We Use Microcontrollers

There are several practical reasons for choosing microcontrollers in projects like this:

- Cost-Effectiveness: A microcontroller board costs much less than using a full computer for a simple dedicated task. Our entire system was built with components that cost less than a normal calculator.
- Energy Efficiency: Microcontrollers consume very little power, which makes them suitable for battery-powered devices. The STM32F446RE we used can also enter low-power sleep modes when nothing is happening.
- Real-Time Response: Because the microcontroller is dedicated to one task, it can react to events almost instantly. This is critical for our project because we need to capture the exact moment a sensor is triggered. Even a 1 ms delay would affect the speed measurement.
- Compact Size: All the components are integrated, so the final device can

be made small and portable.

Why We Chose STM32 Instead of Arduino

At the beginning of the project we discussed which platform to use. Arduino Uno would have been simpler because we already used it in earlier courses, but we decided to go with the STM32F446RE for several reasons. First, the STM32 is a much more powerful chip 180 MHz vs 16 MHz, 128 KB RAM vs 2 KB for the same price as an Arduino Uno. This becomes important when we need to keep historical measurement data, draw histograms, and run a menu system at the same time. With Arduino Uno, the SH1107 OLED display alone uses around half of the available RAM just for the framebuffer. Second, STM32 has multiple hardware timers with input capture mode, which is useful for high-precision time measurement. Even though we ended up using the simpler `millis()` function for our needs, having the option for sub-microsecond timing was a good safety margin.

Another important reason was that the STM32F446RE supports more peripherals at the same time. Our project uses SPI for the OLED display, multiple GPIO interrupts for IR sensors, ADC for the joystick, PWM for the RGB LED and the buzzer, and flash-emulated EEPROM for persistent storage. Doing all this on Arduino Uno would have been very tight on resources.

Problem Solving and Selection of Methods

To solve the speed measurement problem, we used a time-of-flight based methodology. The basic idea is simple: if we know the distance between two sensors and we can measure the time it takes for an object to travel from one to the other, then the speed is just distance divided by time. This is the well-known $V = d/\Delta t$ formula from basic physics.

We chose this method because it is simple, reliable and does not require expensive components like Doppler radar modules. We programmed the microcontroller using PlatformIO with the `Arduino_Core_STM32` framework. We used Visual Studio Code as our editor. The reason we chose this combination is that it gave us access to ready-made libraries (such as `Adafruit_SH110X` for the OLED) while still letting us use the underlying STM32 HAL functions for more advanced things like flash-emulated EEPROM and direct register access when needed. The speed values are calculated and displayed in cm/s by default, but the user can switch to km/h from the settings menu.

CHAPTER III

3.1 CIRCUIT DESIGN

Theoretical Explanation of Components

When designing the circuit we tried to choose components that work well together and that we already had some experience with. The complete component list is given in the table below, and the working principle of each main component is explained after.

Component	Purpose / Notes
STM32 Nucleo F446RE	Main controller. 180 MHz Cortex-M4, runs all logic.
2× HW-201 IR sensor	Detects when an object passes. Adjustable sensitivity via on-board potentiometer.
Waveshare 1.3" OLED (SH1107)	128×64 monochrome display. Connected via SPI.
Funduino Joystick Shield V1.A	Provides analog joystick (X/Y/press) and 4 push buttons (A, B, C, D, F). Voltage switch set to 3.3 V.
MH-FMD passive buzzer	Audio feedback. Driven with PWM through tone() function.
Common-cathode RGB LED	Visual feedback. Three PWM channels (R/G/B) for different colors based on speed.
Perfboard + jumper wires	Used for connecting the modules and providing 3.3 V / GND rails to all components.

Component Details

The Brain (Microcontroller): The STM32F446RE on the Nucleo-64 board acts as the brain of the system. It has more than enough digital pins for our sensors, hardware SPI for the OLED, multiple PWM-capable pins for the LED and buzzer, and analog inputs for the joystick. We powered the board through its USB connection, which also provides the regulated 3.3 V used by the rest of the circuit.

Sensors: For object detection we used two HW-201 reflective infrared sensor modules. Each module has its own IR LED and an IR phototransistor side by side, plus an LM393 comparator and a small adjustment potentiometer on the PCB. When an object reflects the IR light back into the phototransistor, the comparator output (DOUT pin) goes from HIGH to LOW. We adjusted the on-board potentiometers so that both sensors trigger at approximately the same distance, which is important for symmetrical measurements.

The Interface (Display): We used a 1.3-inch Waveshare OLED module with the SH1107 controller. The IM jumper on the back of the module is set to 0, which configures it for SPI mode. This gives us much faster screen updates compared to I2C, which is useful for showing menus, statistics and speed in real time. The display runs at 128×64 pixels

in landscape orientation (we use `setRotation(1)` in the code).

Visual & Audio Feedback: A common-cathode RGB LED is used to give an at-a-glance status indication. The colors mean: green for menu navigation, white for under speed limit, hardware test/statistics screen, blue for a single flash when an IR sensor is triggered, red when the speed exceeds the limit, and orange when it is close to the limit. The MH-FMD passive buzzer plays different tones for menu confirmation, sensor triggers and the over-limit alarm.

Pin Assignment

The pin assignment was carefully chosen to avoid conflicts between the joystick shield and the rest of the circuit. The shield occupies D2–D5 (buttons), D8 (joystick press), A0–A1 (joystick X/Y), so we used the remaining pins for everything else.

Function	Arduino pin	STM32 pin
OLED MOSI (DIN)	D11	PA7
OLED SCK (CLK)	D13	PA5
OLED CS	D10	PB6
OLED DC	D7	PA8
OLED RST	D6	PB10
Joystick X	A0	PA0
Joystick Y	A1	PA1
Joystick press	D8	PA9
Button A / B / C / D	D2 / D3 / D4 / D5	PA10 / PB3 / PB5 / PB4
IR sensor 1	A4	PC1
IR sensor 2	A3	PB0
Buzzer	D9	PC7
RGB LED R/G/B	-	PC9 / PC8 / PC6
F (reset/power) button	D15	PB8

Working Principle and Circuit Logic

We placed the two IR sensors in a straight line at a fixed distance of 20 cm from each

other (this is also adjustable from the settings menu, anywhere between 5 cm and 100 cm). When an object passes through the detection field of the first sensor, its DOUT pin transitions from HIGH to LOW (a falling edge). The microcontroller continuously polls both sensor pins inside the main loop and detects this falling edge by comparing the current value with the previous one. The exact moment when the first edge is detected is captured using the `millis()` function and the system enters the "MEASURING" state. The radar then waits for the second sensor to be triggered. As soon as the second falling edge occurs, the second timestamp is taken and the time difference Δt is calculated.

Once we have Δt , the speed is calculated using:

$$V = d / \Delta t$$

where d is the distance between the two sensors (in cm) and Δt is the elapsed time (in seconds). The result is shown on the OLED display stored in a circular buffer in flash-emulated EEPROM and compared against the speed limit. If the limit is exceeded, the RGB LED turns red and the buzzer alarm is activated for 4 seconds.

One detail we paid attention to is that our detection algorithm is direction-independent. The object can pass either from sensor 1 to sensor 2, or from sensor 2 to sensor 1 both directions give a valid measurement. This is implemented using a small state machine with the variables `radarState` and `firstSensor`.

CHAPTER IV

4.1 METHODOLOGY

Theoretical Explanation of the Algorithm

The core of our software is a time-capture algorithm. Instead of trying to compute the speed continuously, we wait for two discrete events (the two sensor triggers), record the time of each event, and then do a single division at the end. This approach has two advantages: it is computationally light (the microcontroller does almost nothing during the waiting time), and it is much more accurate than any "average speed over time" approach because the two reference points are physically fixed and well-defined.

The algorithm uses a 4-state finite state machine:

- State 0 - IDLE: The system is in the speed radar menu but not actively measuring. Press C to arm.
- State 1 - ARMED: Waiting for the first IR sensor to be triggered. Either sensor can trigger first.

- State 2 - MEASURING: First sensor was triggered, waiting for the second. If too much time passes (4 seconds), the system times out and goes back to IDLE.
- State 3 - RESULT: A valid measurement was just completed. The speed is shown on the screen and the alarm is active if the limit was exceeded. Press C to start a new measurement.

Code Structure and Important Variables

Our code is divided into a few logical parts. The `setup()` function initializes all the pins, loads the saved settings from EEPROM and plays a splash screen while loading. The `loop()` function has two main paths: a "hot path" that runs every loop iteration and only checks the IR sensors for falling edges (so we never miss a trigger), and a "cold path" that runs every 100 ms and handles the user interface, screen drawing and button events. This separation is important because drawing on the OLED takes around 10–20 ms, and if we did it on every loop the IR detection would be too slow.

The most important variables are:

- `t1, t2` - timestamps in milliseconds when the first and second IR sensors are triggered.
- `lastSpeed` - the last calculated speed in cm/s.
- `sensorDistanceCm` - the configurable distance between the two sensors (default 20 cm, range 5-100 cm).
- `speedLimitCms` - the configurable speed limit (default 30 cm/s, range 5-300 cm/s). Above this value the alarm is triggered.
- `radarState, firstSensor` - variables that hold the current state of the measurement state machine.
- `saveNum, saveIndex` - counter and pointer for the circular buffer that stores the last 100 measurements in EEPROM.
- `ledSetting, buzzerSetting, speedUnitSetting` - user preferences (LED on/off, buzzer on/off, cm/s vs km/h). These are saved in EEPROM and loaded at startup.

Persistent Storage with Flash-Emulated EEPROM

Since the STM32F446RE does not have a real EEPROM peripheral, we used the flash-emulated EEPROM provided by the `Arduino_Core_STM32` framework. The trick here is that flash memory has a limited number of erase cycles (around 10,000), so we should not write to it on every measurement. To handle this, we use a buffered approach: at startup we call `eeeprom_buffer_fill()` once to load the entire EEPROM page into RAM. After that, all reads and writes happen in RAM (very fast, no wear), and we only call `eeeprom_buffer_flush()` to physically write the data when the user exits the speed radar or settings menu. This way the actual flash is written only a few times per session instead of hundreds of times.

User Interface - Menu System

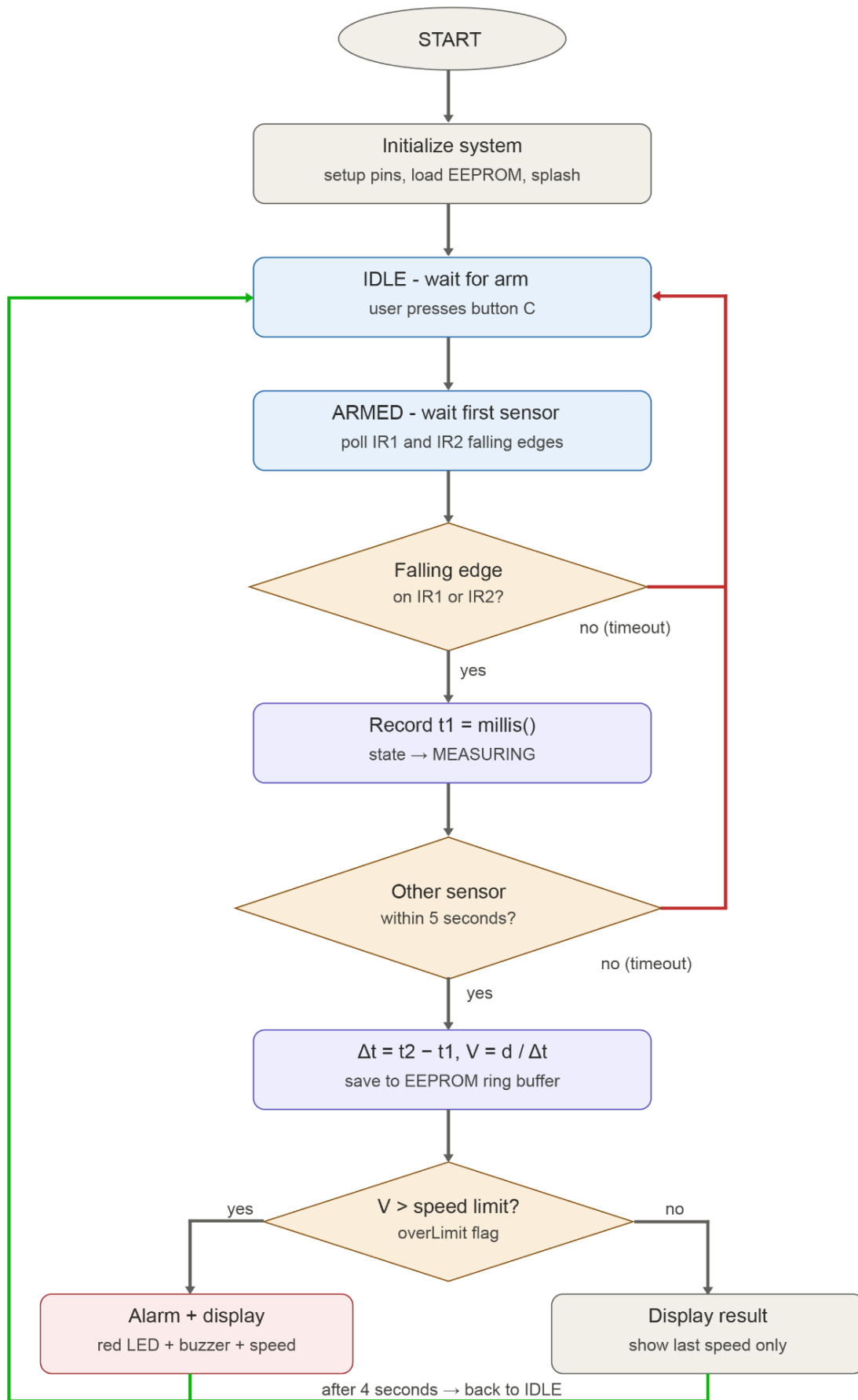
The user interface is organized as a main menu with four items:

- SPEED RADAR - the main measurement screen.
- STATISTICS - shows total count, average speed, maximum speed and the number of over-limit events. The user can press button A to switch between text view and a histogram graph.
- HARDWARE TEST - shows the live values of all inputs (joystick X/Y/press, all 4 buttons, both IR sensors). This screen was very useful during development for verifying connections.
- SETTINGS - six configurable parameters: sensor distance, speed limit, speed unit, LED on/off, buzzer on/off, and a memory reset option.

Navigation works like this: in the main menu, the joystick Y axis moves the cursor up and down, and button C enters the highlighted menu. Inside any sub-menu, button B always goes back. In the settings menu, joystick X is used to change the value of slider-type settings (distance and limit), and button C toggles the on/off-type settings. Each interaction also gives a short audio confirmation through the buzzer. We added F button on PB8 to give the device a hardware-style reset and a low-power off state, similar to a real consumer product. The behavior depends on how long the button is held. Short press is software reset. The function `NVIC_SystemReset()` is called, which restarts the microcontroller. Useful for recovering from any unexpected state. Long press makes the device enters an "off" mode. The OLED is turned off, all LEDs and the buzzer are silenced, and the main loop just waits for the next long press. Pressing F long again brings the device back to life.

Algorithm Flowchart

A simplified flowchart of the speed measurement algorithm is shown below:



Step-by-Step Explanation

1. Initialization: The `setup()` function configures all pins, loads previous settings from EEPROM, and plays the splash animation.
2. Main menu: The user navigates with the joystick and selects "SPEED RADAR" with button C.
3. Arming: Inside the radar menu, pressing C arms the system. The state changes from IDLE to ARMED.
4. First detection: The hot path of the loop continuously checks for falling edges on IR1 and IR2. Whichever fires first becomes "firstSensor" and `t1` is recorded.
5. Second detection: The system waits for the OTHER sensor (not the same one twice) to fire. When it does, `t2` is recorded and the speed is calculated.
6. Display and feedback: The result is shown on the OLED, the LED and buzzer give visual/audio feedback, and the value is added to the EEPROM circular buffer for future statistics.
7. Repeat: Pressing C again arms the system for a new measurement.

CHAPTER V

5.1 RESULTS AND DISCUSSION

5.1 RESULTS AND DISCUSSION

System Results and Interpretations

After we finished the wiring, the firmware and the calibration of the IR sensors, we ran several tests to evaluate the system. For most of our tests we used a Hot Wheels die-cast toy car released down a small inclined ramp, because this gives us a repeatable speed each time (since $v = \sqrt{2gh}$ for an ideal frictionless rolling object). The system successfully detected the toy car at both sensor positions and calculated its speed every time, with results appearing on the OLED almost instantly after the second sensor was triggered.

A typical test session looked like this: we set the sensor distance to 20 cm and the speed limit to 30 cm/s. We let the car move from a fixed point on a low inclined ramp and recorded the value displayed on the screen. After 5-10 runs the average results were as follows:

Test #	Δt (ms)	Calculated Speed (cm/s)	In km/h
1	720	27.8	1.00
2	630	31.7	1.14
3	760	26.3	0.95
4	590	33.9	1.22
5	685	29.2	1.05
Avg.	677	29.8	1.07

The standard deviation of the measured speed is around ± 0.6 cm/s, which corresponds to an error of about ± 2 % across runs. We consider this very acceptable for our setup especially since most of the variation actually comes from the toy car itself such as slightly different starting position, small bumps on the ramp surface, and friction losses, rather than from the electronics.

Performance Evaluation

We evaluated the performance based on three main criteria:

- **Accuracy:** The system gives consistent results within ± 2 % across repeated trials, which matches our expectation for a millis()-based timing system. Higher precision would require microsecond-level timing (using STM32 hardware timers in input capture mode), but for our use case this was not necessary.
- **Response time:** The OLED is updated within 100 ms of the measurement, and the alarm (LED + buzzer) is activated within the same window. From the user's perspective, the result appears "instantly".
- **Robustness:** Thanks to the falling-edge detection and a 1.5-second cooldown after each measurement, the system does not produce false multi-triggers when an object is sitting still in front of a sensor.

Conditions and Limitations

During testing we observed the following conditions and limitations:

- **Best conditions:** A medium-sized object passing close (within 5–10 cm) to both sensors at a moderate speed gives the most reliable results. The IR sensors must be calibrated to roughly the same trigger distance using their on-board potentiometers.

- Object size: If the object is too small (smaller than the IR detection cone) or has a very dark, non-reflective surface, the sensors may not trigger reliably.
- Maximum speed: Because we use `millis()` (1 ms resolution), the practical maximum measurable speed depends on the sensor distance. With a 20 cm separation, even a 10 m/s (36 km/h) object would still take 20 ms, well above the resolution limit. So speed itself was not the limiting factor, sensor response time was more important.
- Ambient light: HW-201 modules use IR reflection, so very strong sunlight or IR-rich light sources nearby can affect the detection threshold. In normal indoor lighting we did not see this problem.

Advantages and Disadvantages

Advantages:

- Low cost - the entire bill of materials is under 2000 TL.
- Real-time display with built-in user interface, no laptop or external software needed.
- Persistent settings and history, measurements are stored in non-volatile memory and survive reboots.
- Multiple operating modes (radar, statistics, hardware test, settings) accessible from a single menu.
- Direction-independent measurement, the object can pass in either direction.

Disadvantages:

- Sensor alignment is critical. If the two sensors are not parallel and at the same height, the effective distance d is wrong, which directly affects the speed calculation.
- HW-201 modules are reflective-type their effective range is about 2-30 cm, which limits how far the moving object can be from the device.
- Small or dark objects may not trigger the sensors reliably.
- Time resolution is limited to 1 ms by the `millis()` function; for ultra-high-speed objects (e.g. real cars on a road), microsecond-level timing would be needed.

5.2 Realistic Design Constraints and Considerations

Cost / Economy

Cost was an important constraint for us as students. We tried to use components that are cheap and easy to find. The most expensive item is the STM32 Nucleo board itself (around 1600 TL), but everything else (HW-201 sensors, OLED, joystick shield, RGB

LED, buzzer, prefboard, jumper wires) together costs less than 400 TL. So the total project cost is well under 2000 TL, much less than a commercial speed measurement device, which would cost thousands of liras for industrial-grade hardware.

We deliberately chose HW-201 IR modules instead of more advanced solutions like Doppler radar (HB100) or laser time-of-flight sensors. While those would give better range and accuracy, they would also increase the cost by 5–10× and add a lot of signal processing complexity (FFT, amplification, filtering) that goes beyond the scope of this course.

Safety and Health

Even though our project is small, we considered safety from the beginning. The entire system runs on 3.3 V supplied by the Nucleo board's onboard regulator (which itself is fed from 5 V USB). At these voltages there is no risk of electric shock. We also made sure that no component runs hot during operation, we measured the surface temperature of the Nucleo and it stayed under 35 °C even after several hours of continuous use.

Another safety consideration is sensor placement. We made sure the IR sensors and other components are placed in a stable position so that nothing comes loose during testing, and so that the moving object (the toy car) cannot collide with any wires or modules. The buzzer volume was also kept moderate so it does not startle anyone in the lab.

Manufacturability / Implementability

We wanted our design to be easy to assemble and modify. By using ready-made breakout modules (the Nucleo board, the joystick shield, the OLED module and the HW-201 sensors), the wiring is simplified to plugging headers into Nucleo board connecting jumper wires. There is no soldering needed except for V/GND rails.

Choosing SPI for the OLED was also a manufacturability decision. Even though I2C would have used fewer wires, SPI is significantly faster (especially for a 128×64 framebuffer), which makes the splash animation and histogram rendering smooth. The 3 extra wires (CS, DC, RST) were a worthwhile trade-off.

The use of EEPROM-buffered storage means that user settings persist across reboots without requiring any external hardware (no real EEPROM chip, no SD card). This makes the design simpler and reduces the chance of failure points.

5.3 Conclusion

In conclusion, this project gave us a real opportunity to put together everything we have learned about microcontrollers, sensors, displays and embedded software design into a single working device. We successfully built a speed radar that can measure the velocity

of a passing object with about $\pm 2\%$ accuracy, display the result on a 128×64 OLED, store up to 100 measurements in non-volatile memory, and provide a complete user interface with multiple menus and configurable settings.

Throughout the process we learned several important lessons. First, the importance of separating fast and slow logic in a real-time system, our hot path / cold path structure is what allows us to never miss an IR trigger while still updating a complex graphical UI. Second, the importance of persistent storage management, we had to carefully use the buffered EEPROM API to avoid wearing out the flash memory. Third, the value of incremental development and good debugging tools, the HARDWARE TEST menu we built early on saved us many hours by letting us check every input visually.

The strengths of our project are its low cost, complete on-device user experience, and the fact that it really demonstrates how a basic physics formula ($V = d/\Delta t$) can be implemented end-to-end in an embedded system. Its weaknesses are mainly related to the IR sensor type, the limited range and sensitivity to alignment make it more suitable for laboratory-scale demonstrations than for actual traffic.

In the future, this project could be improved in several ways: replacing the HW-201 sensors with laser time-of-flight or actual Doppler radar modules for better range and accuracy, adding an SD card for long-term data logging, or using STM32's hardware timer input capture mode for nanosecond-level timing precision. Wireless connectivity (Bluetooth or WiFi via an ESP32 module) could also be added so the data can be visualized on a smartphone in real time. Overall, we are happy with what we built, it is genuinely useful, looks polished, and gave each member of the team something concrete to learn.



REFERENCES

- [1] NUCLEO-F446RE Official Documentation and Reference, https://github.com/stm32duino/Arduino_Core_STM32/wiki/API
- [2] NUCLEO-F446RE Pinout Guide, <https://os.mbed.com/platforms/ST-Nucleo-F446RE/>
- [3] Joystick Shield Datasheet and User Manual, <https://handsontec.com/dataspecs/module/Arduino%20Shield/Joystick%20Shield.pdf> .
- [4] Arduino Official Documentation and Reference, <https://docs.arduino.cc/language-reference/>
- [5] Adafruit_SH110xOfficial Documentation and Reference, https://github.com/adafruit/Adafruit_SH110X

APPENDIX-CODE

APPENDIX-CODE

```
#include <Arduino.h>

#include <SPI.h>

#include <Adafruit_GFX.h>
#include <Adafruit_SH110X.h>
#include <EEPROM.h>

// =====

// SPEED RADAR

// 4 buttons (A/B/C/D) + joystick + OLED + buzzer + IR

// =====

// OLED SPI pins

#define OLED_DC    PA8
#define OLED_RST   PB10
#define OLED_CS    PB6
```

```
Adafruit_SH1107 display = Adafruit_SH1107(64, 128, &SPI, OLED_DC,
OLED_RST, OLED_CS);

// Joystick + buttons

#define JOY_X      A0      // PA0
#define JOY_Y      A1      // PA1
#define JOY_SW     PA9     // D8
#define BTN_A      PA10    // D2
#define BTN_B      PB3     // D3
#define BTN_C      PB5     // D4
#define BTN_D      PB4     // D5
#define BTN_F      PB8     // D15 -> reset button

// IR sensors

#define IR1         PC1     // A4
#define IR2         A3      // A3

#define DEFAULT_SENSOR_DISTANCE_CM  20.0f
#define MIN_SENSOR_DISTANCE_CM      5.0f
#define MAX_SENSOR_DISTANCE_CM      100.0f
#define SENSOR_DISTANCE_STEP_CM     1.0f
#define DEFAULT_SPEED_LIMIT_CMS     30.0f
#define MIN_SPEED_LIMIT_CMS         5.0f
#define MAX_SPEED_LIMIT_CMS         300.0f
#define SPEED_LIMIT_STEP_CMS        5.0f

#define MEASURE_TIMEOUT_MS  5000

#define COOLDOWN_MS         1500    // Do not accept a new measurement
for 1.5s after a measurement

#define F_LONG_PRESS_MS     1500
#define F_DEBOUNCE_MS       40

// Buzzer

#define BUZZER      PC7      // D9
```

```
//LED

#define RED      PC9      // D13
#define GREEN    PC8      // D13
#define BLUE     PC6      // D13

unsigned long lastUpdate = 0;
unsigned long lastBeep = 0;

bool lastBtnC = false;
bool lastBtnB = false;
bool lastBtnA = false;

bool systemOff = false;
bool fWasDown = false;
bool fLongHandled = false;
unsigned long fPressStarted = 0;

int statsView = 0;    // 0 = text statistics, 1 = histogram graph (toggle
with A)

int menuSelect = 0;
int currentMenu = -1;
int maxMenu = 3;      // 0=SPEED RADAR, 1=STATISTICS, 2=HARDWARE TEST,
3=SETTINGS

// SPEED RADAR state
unsigned long t1 = 0;      // 1st sensor trigger time
unsigned long t2 = 0;      // 2nd sensor trigger time
float lastSpeed = 0.0;     // last measured speed (cm/s)
bool overLimit = false;    // did the speed exceed the limit?
```

```

bool lastIr1 = HIGH;          // previous value for edge detection
bool lastIr2 = HIGH;

int firstSensor = 0;          // 0=none, 1=IR1 first, 2=IR2 first
int radarState = 0;           // 0=IDLE, 1=ARMED, 2=MEASURING, 3=RESULT
unsigned long resultTime = 0; // time captured when entering RESULT (for
4s LED/buzzer cap)

unsigned long sensorFlashUntil = 0; // blue LED flash end time when a
sensor is triggered

#define ALARM_DURATION_MS 4000
#define SENSOR_FLASH_MS 80

// EEPROM layout:
//   offset 0..3   : saveNum          (total record count, 0..100)
//   offset 4..7   : saveIndex        (next write position, 0..99)
//   offset 8..407 : records           (100 floats, 4 bytes each)
//   offset 408    : ledSetting        (4 byte int)
//   offset 412    : buzzerSetting     (4 byte int)
//   offset 416    : sensorDistanceCm (4 byte float)
//   offset 420    : speedLimitCms     (4 byte float)
//   offset 424    : speedUnitSetting  (4 byte int, 0=cm/s, 1=km/h)

int saveNum = 0;              // total record count (stays fixed after reaching
max)

int saveIndex = 0;           // ring buffer write position

const int maxSave = 100;

#define EEPROM_RECORDS_OFFSET 8
#define EEPROM_LED_OFFSET 408
#define EEPROM_BUZZER_OFFSET 412
#define EEPROM_DISTANCE_OFFSET 416
#define EEPROM_SPEED_LIMIT_OFFSET 420

```

```

#define EEPROM_SPEED_UNIT_OFFSET 424

int settingSelect = 0;
int currentSetting = 0;
int maxSetting = 5;    // 0=Distance, 1=Limit, 2=Unit, 3=LED, 4=BUZZER,
5=Reset
int ledSetting = 1;
int buzzerSetting = 1;
float sensorDistanceCm = DEFAULT_SENSOR_DISTANCE_CM;
float speedLimitCms = DEFAULT_SPEED_LIMIT_CMS;
int speedUnitSetting = 0;

// EEPROM buffer
void bufWriteFloat(uint32_t addr, float value) {
    uint8_t* p = (uint8_t*)&value;
    for (int i = 0; i < 4; i++) eeprom_buffered_write_byte(addr + i, p[i]);
}
float bufReadFloat(uint32_t addr) {
    float value;
    uint8_t* p = (uint8_t*)&value;
    for (int i = 0; i < 4; i++) p[i] = eeprom_buffered_read_byte(addr + i);
    return value;
}
void bufWriteInt(uint32_t addr, int value) {
    uint8_t* p = (uint8_t*)&value;
    for (int i = 0; i < 4; i++) eeprom_buffered_write_byte(addr + i, p[i]);
}
int bufReadInt(uint32_t addr) {
    int value;
    uint8_t* p = (uint8_t*)&value;

```

```
    for (int i = 0; i < 4; i++) p[i] = eeprom_buffered_read_byte(addr + i);
    return value;
}

float clampFloat(float value, float minValue, float maxValue) {
    if (value < minValue) return minValue;
    if (value > maxValue) return maxValue;
    return value;
}

float speedForDisplay(float speedCms) {
    return (speedUnitSetting == 1) ? (speedCms * 0.036f) : speedCms;
}

const char* speedUnitLabel() {
    return (speedUnitSetting == 1) ? "km/h" : "cm/s";
}

int speedDecimals() {
    return (speedUnitSetting == 1) ? 2 : 1;
}

void printSpeed(float speedCms) {
    display.print(speedForDisplay(speedCms), speedDecimals());
    display.print(" ");
    display.print(speedUnitLabel());
}

void drawSlider(int x, int y, int w, int h, float value, float minValue,
float maxValue) {
```

```

display.drawRect(x, y, w, h, SH110X_WHITE);

float ratio = (value - minValue) / (maxValue - minValue);

if (ratio < 0.0f) ratio = 0.0f;

if (ratio > 1.0f) ratio = 1.0f;

int fillW = (int)((w - 2) * ratio);

if (fillW > 0) display.fillRect(x + 1, y + 1, fillW, h - 2,
SH110X_WHITE);
}

void setRGB(uint8_t r, uint8_t g, uint8_t b) {

    if (ledSetting == 1) {

        analogWrite(RED, r);

        analogWrite(GREEN, g);

        analogWrite(BLUE, b);

    } else {

        analogWrite(RED, 0);

        analogWrite(GREEN, 0);

        analogWrite(BLUE, 0);

    }

}

void forceOutputsOff() {

    noTone(BUZZER);

    analogWrite(RED, 0);

    analogWrite(GREEN, 0);

    analogWrite(BLUE, 0);

}

void softResetFromF() {

    Serial.println("[RESET] F short press - resetting board");

```

```
    eeprom_buffer_flush();

    forceOutputsOff();

    delay(100);

    NVIC_SystemReset();
}

void enterOffMode() {

    Serial.println("[POWER] F long press - off mode");

    eeprom_buffer_flush();

    forceOutputsOff();

    radarState = 0;

    firstSensor = 0;

    currentMenu = -1;

    display.clearDisplay();

    display.display();

    display.oled_command(SH110X_DISPLAYOFF);

    systemOff = true;
}

void exitOffMode() {

    Serial.println("[POWER] F long press - on mode");

    systemOff = false;

    lastIr1 = digitalRead(IR1);

    lastIr2 = digitalRead(IR2);

    lastUpdate = 0;

    display.oled_command(SH110X_DISPLAYON);

    display.clearDisplay();
}
```



```
display.display();

if (buzzerSetting == 1) tone(BUZZER, 1200, 80);
}

void handleFButton() {

    bool fDown = !digitalRead(BTN_F);

    unsigned long now = millis();

    if (fDown && !fWasDown) {

        fWasDown = true;

        fLongHandled = false;

        fPressStarted = now;

    }

    if (fDown && fWasDown && !fLongHandled &&
        (now - fPressStarted >= F_LONG_PRESS_MS)) {

        fLongHandled = true;

        if (systemOff) {

            exitOffMode();

        } else {

            enterOffMode();

        }

    }

    if (!fDown && fWasDown) {

        unsigned long pressDuration = now - fPressStarted;

        bool wasLongPress = fLongHandled || (pressDuration >=
F_LONG_PRESS_MS);

        fWasDown = false;

    }

}
```

```

        if (!wasLongPress && pressDuration >= F_DEBOUNCE_MS && !systemOff) {

            softResetFromF();

        }

    }

}

void setup() {

    Serial.begin(115200);

    delay(500);

    Serial.println("SPEED RADAR STARTING...");


    pinMode(JOY_SW, INPUT_PULLUP);

    pinMode(BTN_A, INPUT_PULLUP);

    pinMode(BTN_B, INPUT_PULLUP);

    pinMode(BTN_C, INPUT_PULLUP);

    pinMode(BTN_D, INPUT_PULLUP);

    pinMode(BTN_F, INPUT_PULLUP);

    pinMode(IR1, INPUT);

    pinMode(IR2, INPUT);

    pinMode(BUZZER, OUTPUT);

    pinMode(RED, OUTPUT);

    pinMode(GREEN, OUTPUT);

    pinMode(BLUE, OUTPUT);


    digitalWrite(RED, LOW);

    digitalWrite(GREEN, LOW);

    digitalWrite(BLUE, LOW);


    // Fill the EEPROM buffer from flash once, then read from RAM

    eeprom_buffer_fill();

```

```

// Load previous counters; sanity checks clear junk values on first
boot

saveNum    = bufReadInt(0);

saveIndex = bufReadInt(4);

if (saveNum < 0 || saveNum > maxSave)    saveNum = 0;

if (saveIndex < 0 || saveIndex >= maxSave) saveIndex = 0;

// Load settings; boolean settings must be 0 or 1, otherwise use
default 1

ledSetting    = bufReadInt(EEPROM_LED_OFFSET);

buzzerSetting = bufReadInt(EEPROM_BUZZER_OFFSET);

sensorDistanceCm = bufReadFloat(EEPROM_DISTANCE_OFFSET);

speedLimitCms = bufReadFloat(EEPROM_SPEED_LIMIT_OFFSET);

speedUnitSetting = bufReadInt(EEPROM_SPEED_UNIT_OFFSET);

if (ledSetting != 0 && ledSetting != 1)    ledSetting = 1;

if (buzzerSetting != 0 && buzzerSetting != 1) buzzerSetting = 1;

if (!(sensorDistanceCm >= MIN_SENSOR_DISTANCE_CM && sensorDistanceCm <=
MAX_SENSOR_DISTANCE_CM)) {

    sensorDistanceCm = DEFAULT_SENSOR_DISTANCE_CM;

}

if (!(speedLimitCms >= MIN_SPEED_LIMIT_CMS && speedLimitCms <=
MAX_SPEED_LIMIT_CMS)) {

    speedLimitCms = DEFAULT_SPEED_LIMIT_CMS;

}

if (speedUnitSetting != 0 && speedUnitSetting != 1) speedUnitSetting =
0;

Serial.print("[EEPROM] saveNum="); Serial.print(saveNum);

Serial.print(" saveIndex="); Serial.print(saveIndex);

Serial.print(" LED="); Serial.print(ledSetting);

Serial.print(" BUZZER="); Serial.print(buzzerSetting);

Serial.print(" DIST="); Serial.print(sensorDistanceCm);

Serial.print(" LIMIT="); Serial.print(speedLimitCms);

Serial.print(" UNIT="); Serial.println(speedUnitSetting);

```

```
display.begin(0, true);
display.setRotation(1);
display.clearDisplay();
display.display();

// Splash frame
auto drawSplashFrame = []() {
    display.clearDisplay();
    display.setTextColor(SH110X_WHITE);
    display.setTextSize(1);

    display.setCursor(34, 0);
    display.print("SPEED RADAR");

    // "STARTING"
    display.setCursor(34, 10);
    display.print("STARTING");

    int dotCount = (millis() / 400) % 4;
    for (int i = 0; i < dotCount; i++) display.print(".");

    // Radar
    const int cx = 64, cy = 42, r = 18;
    display.drawCircle(cx, cy, r, SH110X_WHITE);
    display.drawCircle(cx, cy, (r * 2) / 3, SH110X_WHITE);
    display.drawCircle(cx, cy, r / 3, SH110X_WHITE);
    display.drawPixel(cx, cy, SH110X_WHITE);

    // Sweep line
    float angle = (millis() % 1500) / 1500.0f * 2.0f * PI;
```

```

    int x2 = cx + (int)(r * cos(angle));

    int y2 = cy + (int)(r * sin(angle));

    display.drawLine(cx, cy, x2, y2, SH110X_WHITE);

    // Trail

    for (int i = 1; i <= 3; i++) {

        float ta = angle - (i * 0.18f);

        if (ta < 0) ta += 2.0f * PI;

        int tr = r - i * 4;

        if (tr > 0) {

            int tx = cx + (int)(tr * cos(ta));

            int ty = cy + (int)(tr * sin(ta));

            display.drawPixel(tx, ty, SH110X_WHITE);

        }

    }

    display.display();

};

// Play the animation while waiting instead of using delay
auto splashDelay = [&drawSplashFrame](unsigned long ms) {

    unsigned long start = millis();

    while (millis() - start < ms) {

        drawSplashFrame();

        delay(20);

    }

};

// Intro music + radar sweep animation
tone(BUZZER, 660, 100); splashDelay(150);

```

```
tone(BUZZER, 660, 100); splashDelay(150);
tone(BUZZER, 1318, 100); splashDelay(150);
tone(BUZZER, 660, 100); splashDelay(150);
tone(BUZZER, 660, 100); splashDelay(150);
tone(BUZZER, 1174, 100); splashDelay(150);
tone(BUZZER, 660, 100); splashDelay(150);
tone(BUZZER, 660, 100); splashDelay(150);
tone(BUZZER, 1046, 100); splashDelay(150);
tone(BUZZER, 660, 100); splashDelay(150);
tone(BUZZER, 660, 100); splashDelay(150);
tone(BUZZER, 932, 100); splashDelay(150);
tone(BUZZER, 660, 100); splashDelay(150);
tone(BUZZER, 660, 100); splashDelay(150);
tone(BUZZER, 987, 100); splashDelay(150);
tone(BUZZER, 1046, 100); splashDelay(150);
tone(BUZZER, 660, 100); splashDelay(150);
tone(BUZZER, 660, 100); splashDelay(150);
tone(BUZZER, 1318, 100); splashDelay(150);
tone(BUZZER, 660, 100); splashDelay(150);
tone(BUZZER, 660, 100); splashDelay(150);
tone(BUZZER, 1174, 100); splashDelay(150);
tone(BUZZER, 660, 100); splashDelay(150);
tone(BUZZER, 660, 100); splashDelay(150);
tone(BUZZER, 1046, 100); splashDelay(150);
tone(BUZZER, 660, 100); splashDelay(150);
tone(BUZZER, 660, 100); splashDelay(150);
tone(BUZZER, 932, 500); splashDelay(150);

splashDelay(2000);
```

```

}

void loop() {
    handleFButton();

    if (systemOff) {
        forceOutputsOff();

        delay(10);

        return;
    }

    // HOT Path

    bool ir1Now = digitalRead(IR1);

    bool ir2Now = digitalRead(IR2);

    bool ir1Falling = (lastIr1 == HIGH && ir1Now == LOW); // 1->0
transition
    bool ir2Falling = (lastIr2 == HIGH && ir2Now == LOW);

    lastIr1 = ir1Now;

    lastIr2 = ir2Now;

    // Blue LED: one 80ms flash when an object is detected. The latch
prevents a second trigger and resets only after both sensors are really
HIGH, with a short stable filter against bouncing.

    static bool blueLatch = false;

    static unsigned long bothHighSince = 0;

    if (ir1Now == HIGH && ir2Now == HIGH) {
        if (bothHighSince == 0) bothHighSince = millis();

        if (millis() - bothHighSince > 150) blueLatch = false;
    } else {
        bothHighSince = 0;
    }

    if ((ir1Now == LOW || ir2Now == LOW) && !blueLatch) {

```

```

    sensorFlashUntil = millis() + 80;

    blueLatch = true;
}

// Measure only while the speed radar menu is open
if (currentMenu == 0) {

    // Helper that completes a measurement
    auto finishMeasurement = [&](const char* dirLabel) {
        t2 = millis();

        unsigned long delta = (t2 > t1) ? (t2 - t1) : 1;

        lastSpeed = sensorDistanceCm / (delta / 1000.0f);

        overLimit = (lastSpeed > speedLimitCms);

        resultTime = millis();

        radarState = 3;          // RESULT: C is required for a new
measurement

        Serial.print("[RADAR] DONE "); Serial.print(dirLabel);
        Serial.print(" delta="); Serial.print(delta);
        Serial.print("ms speed="); Serial.println(lastSpeed);

        //Put the new speed into the current write slot in the RAM buffer
        bufWriteFloat(EEPROM_RECORDS_OFFSET + saveIndex * 4, lastSpeed);
        //Advance the ring buffer index (wrap from 99 to 0)
        saveIndex = (saveIndex + 1) % maxSave;

        //Increase the counter (stays fixed after reaching max)
        if (saveNum < maxSave) saveNum++;

        //Write the counter and index into the RAM buffer too
        bufWriteInt(0, saveNum);
        bufWriteInt(4, saveIndex);

        //Flash transfer (eeprom_buffer_flush) is done when exiting the
menu

```



```

};

// IR1 falling edge
if (ir1Falling) {
    if (radarState == 1) {
        // ARMED: IR1 triggered as the first sensor
        t1 = millis();
        firstSensor = 1;
        radarState = 2;
        Serial.print("[RADAR] IR1 first t1="); Serial.println(t1);
    } else if (radarState == 2 && firstSensor == 2) {
        // MEASURING: IR2 fell first, IR1 is second; finish the
measurement
        finishMeasurement("IR2->IR1");
    }
    // Ignore IR1 in other states (IDLE/RESULT or the same sensor
again)
}

// IR2 falling edge
if (ir2Falling) {
    if (radarState == 1) {
        // ARMED: IR2 triggered as the first sensor
        t1 = millis();
        firstSensor = 2;
        radarState = 2;
        Serial.print("[RADAR] IR2 first t1="); Serial.println(t1);
    } else if (radarState == 2 && firstSensor == 1) {
        // MEASURING: IR1 fell first, IR2 is second; finish the
measurement
        finishMeasurement("IR1->IR2");
    }
}

```

```

    }

    // Ignore IR2 in other states (IDLE/RESULT or the same sensor
again)

    }

    // Timeout: the second sensor did not trigger within
MEASURE_TIMEOUT_MS

    if (radarState == 2 && (millis() - t1 > MEASURE_TIMEOUT_MS)) {

        Serial.println("[RADAR] TIMEOUT");

        radarState = 0;          // Return to IDLE and wait for C again

        firstSensor = 0;

    }

}

// COLD PATH: UI/log every 100ms
if (millis() - lastUpdate > 100) {

    lastUpdate = millis();

    int    jx    = analogRead(JOY_X);
    int    jy    = analogRead(JOY_Y);
    bool   jsw    = !digitalRead(JOY_SW);
    bool   btnA   = !digitalRead(BTN_A);
    bool   btnB   = !digitalRead(BTN_B);
    bool   btnC   = !digitalRead(BTN_C);
    bool   btnD   = !digitalRead(BTN_D);
    bool   ir1    = digitalRead(IR1);
    bool   ir2    = digitalRead(IR2);

    bool   cPressed = btnC && !lastBtnC;
    bool   bPressed = btnB && !lastBtnB;
    bool   aPressed = btnA && !lastBtnA;

```

```

// A toggles graph/text in the STATISTICS menu
if (currentMenu == 1 && aPressed) {
    statsView = 1 - statsView;

    if (buzzerSetting == 1) tone(BUZZER, 1500, 50);
}

// Enter the selected menu when C is newly pressed
if (currentMenu == -1 && cPressed) {
    currentMenu = menuSelect;

    cPressed = false;

    if (currentMenu == 0) {
        // Reset the state when entering SPEED RADAR

        radarState = 0;

        firstSensor = 0;

        t1 = 0;

        lastSpeed = 0.0;

        overLimit = false;
    }

    if (buzzerSetting == 1) {
        tone(BUZZER, 1200, 80);
    }
}

// C starts a new measurement while inside SPEED RADAR
else if (currentMenu == 0 && cPressed) {
    if (radarState == 0 || radarState == 3) {
        radarState = 1;          // ARMED: wait for the first sensor

        firstSensor = 0;

        t1 = 0;

        Serial.println("[RADAR] ARMED - waiting for first sensor");
    }
}

```

```

        if (buzzerSetting == 1) {

            tone(BUZZER, 1200, 80);

        }

    }

}

// Return to the main menu when B is newly pressed

if (currentMenu != -1 && bPressed) {

    // Flush pending changes when exiting radar or settings, instead of
flushing after every change

    if (currentMenu == 0 || currentMenu == 3) {

        eeprom_buffer_flush();

        Serial.print("[EEPROM] flush (menu="); Serial.print(currentMenu);

        Serial.println(" exit)");

    }

    currentMenu = -1;

    if (buzzerSetting == 1) {

        tone(BUZZER, 800, 80);

    }

}

if (jy < 100 && currentMenu == -1 && menuSelect < maxMenu){

    menuSelect = menuSelect + 1;

    delay(150); // Short delay to prevent fast scrolling
}else if (jy > 1000 && currentMenu == -1 && menuSelect > 0){

    menuSelect = menuSelect - 1;

    delay(150); // Short delay to prevent fast scrolling

}

```

```
display.clearDisplay();

display.setCursor(0, 0);

// =====
// MAIN MENU
// =====

if (currentMenu == -1) {

    setRGB(0, 255, 0);

    display.setCursor(24, 0);

    display.println("-- MAIN MENU --");

    display.setCursor(0, 10);

    display.print("SPEED RADAR");

    if (menuSelect == 0) display.println("<");
    else                  display.println(" ");

    display.print("STATISTICS");

    if (menuSelect == 1) display.println("<");
    else                  display.println(" ");

    display.print("HARDWARE TEST");

    if (menuSelect == 2) display.println("<");
    else                  display.println(" ");

    display.print("SETTINGS");

    if (menuSelect == 3) display.println("<");
    else                  display.println(" ");

    display.println();
}
```

```

        display.println("C = Enter");
    }

    else if (currentMenu == 0) {

        display.setCursor(18, 0);

        display.println("-- SPEED RADAR --");

        display.setCursor(0, 10);

        display.print("Status: ");

        if (radarState == 0)          display.println("NOT READY");
        else if (radarState == 1) display.println("WAITING");
        else if (radarState == 2) display.println("MEASURING");
        else                          display.println(overLimit ? "OVER
LIMIT!" : "MEASURED");

        display.println();

        display.print("Last speed: ");

        if (lastSpeed > 0) {
            printSpeed(lastSpeed);

            display.println();
        } else {
            display.println("---");
        }

        display.print("Limit:   ");

        printSpeed(speedLimitCms);

        display.println();

        display.println();

        display.print("IR1:"); display.print(ir1);
    }
}

```

```
display.print(" IR2:"); display.println(ir2);

display.println();

display.println("C = Start");

display.println("B = Back");

// Speed based color

bool resultActive = (radarState == 3 &&
                    (millis() - resultTime) < ALARM_DURATION_MS);

bool sensorFlashActive = (millis() < sensorFlashUntil);

if (sensorFlashActive) {
    setRGB(0, 0, 255);      // blue: single flash
} else if (resultActive) {
    if (lastSpeed > speedLimitCms) {
        setRGB(255, 0, 0);    // red
    } else if (lastSpeed >= (speedLimitCms * 2.0f / 3.0f)) {
        setRGB(255, 30, 0);    // orange
    } else {
        setRGB(255, 255, 255); // white
    }
} else {
    setRGB(0, 0, 0);
}

// Buzzer alarm: 4s when the limit is exceeded

if (resultActive && overLimit) {
    if (buzzerSetting == 1 && (millis() - lastBeep > 500)) {
        tone(BUZZER, 1500, 200);

        lastBeep = millis();
    }
}
```

```

    }

}

else if (currentMenu == 1) {

    setRGB(255, 255, 255);

    // text and graph views use this data
    // Histogram bins: 6 bins, each 10 cm/s wide
    //    [0-10) [10-20) [20-30) [30-40) [40-50) [50+]

    const int BIN_COUNT = 6;

    int counts[BIN_COUNT] = {0,0,0,0,0,0};

    float totalSpeed = 0, maxSpeed = 0;

    int overLimitCount = 0;

    for (int i = 0; i < saveNum; i++) {

        float h = bufReadFloat(EEPROM_RECORDS_OFFSET + i * 4);

        totalSpeed += h;

        if (h > maxSpeed) maxSpeed = h;

        if (h > speedLimitCms) overLimitCount++;

        int bin = (int)(h / 10.0f);

        if (bin < 0) bin = 0;

        if (bin >= BIN_COUNT) bin = BIN_COUNT - 1;

        counts[bin]++;

    }

    float averageSpeed = (saveNum > 0) ? (totalSpeed / saveNum) : 0.0f;

    if (statsView == 0) {

        // TEXT VIEW

        display.setCursor(18, 0);

        display.println("-- STATISTICS --");
    }
}

```



```

display.setCursor(0, 10);

display.print("Total:  "); display.println(saveNum);

display.print("Avg:    ");

if (saveNum > 0) { printSpeed(averageSpeed); display.println(); }
else             { display.println("---"); }

display.print("Max:    ");

if (saveNum > 0) { printSpeed(maxSpeed); display.println(); }
else             { display.println("---"); }

display.print("Over Limit: "); display.println(overLimitCount);


display.println();

display.println("A = Graph B = Back");
} else {

  // HISTOGRAM VIEW

  display.setCursor(0, 0);

  display.print("-- HISTOGRAM (");

  display.print(saveNum); display.println(") --");


  if (saveNum == 0) {

    display.setCursor(0, 24);

    display.println("No records");

  } else {

    // Highest bin count for bar scaling

    int maxCount = 1;

    for (int i = 0; i < BIN_COUNT; i++) {

      if (counts[i] > maxCount) maxCount = counts[i];

    }


    const int baseline = 38; // bar baseline (horizontal line
here)

```

```

const int histHeight = 20;    // maximum bar height

const int binStride   = 21;    // distance between bin centers

const int barWidth    = 18;

// Draw bars

for (int i = 0; i < BIN_COUNT; i++) {

    int barH = (counts[i] * histHeight) / maxCount;

    int x = i * binStride;

    if (barH > 0) {

        display.fillRect(x, baseline - barH, barWidth, barH,
SH110X_WHITE);

    }

}

// Horizontal baseline

display.drawLine(0, baseline, 126, baseline, SH110X_WHITE);

// Limit line, positioned dynamically on the cm/s histogram
scale.

int limitX = (int)((speedLimitCms / 10.0f) * binStride - 2);
if (limitX < 0) limitX = 0;
if (limitX > 126) limitX = 126;

for (int y = baseline - histHeight; y <= baseline; y += 2) {

    display.drawPixel(limitX, y, SH110X_WHITE);

}

}

// X-axis labels (y=42)

display.setCursor(0, 42); display.print("0");
display.setCursor(21, 42); display.print("10");
display.setCursor(42, 42); display.print("20");

```

```

        display.setCursor(63, 42); display.print("30");

        display.setCursor(84, 42); display.print("40");

        display.setCursor(105, 42); display.print("50+");

        // Footer

        display.setCursor(0, 56);

        display.print("A=Text  B=Back");

    }

}

// =====

//  HARDWARE TEST MENU

// =====

else if (currentMenu == 2) {

    setRGB(255, 255, 255);

    display.setCursor(8, 0);

    display.println("-- HARDWARE TEST --");

    display.setCursor(0, 8);

    display.print("JX:");    display.print(jx);

    display.print(" JY:");    display.println(jy);

    display.print("SW:");    display.println(jsw);

    display.print("A:");      display.print(btnA);

    display.print(" B:");      display.print(btnB);

    display.print(" C:");      display.print(btnC);

    display.print(" D:");      display.println(btnD);

    display.print("IR1:");    display.print(ir1);

    display.print(" IR2:");    display.println(ir2);

```

```
display.println();

display.println("D = Back");

display.println("SW = Beep");


Serial.print("JX:"); Serial.print(jx);
Serial.print(" JY:"); Serial.print(jy);
Serial.print(" SW:"); Serial.print(jsw);
Serial.print(" A:"); Serial.print(btnA);
Serial.print(" B:"); Serial.print(btnB);
Serial.print(" C:"); Serial.print(btnC);
Serial.print(" D:"); Serial.print(btnD);
Serial.print(" IR1:"); Serial.print(ir1);
Serial.print(" IR2:"); Serial.println(ir2);
}

else if (currentMenu == 3) {

    setRGB(0, 255, 0);

    if (jy < 100 && settingSelect < maxSetting){
        settingSelect = settingSelect + 1;
        delay(150);
    }else if (jy > 1000 && settingSelect > 0){
        settingSelect = settingSelect - 1;
        delay(150);
    }

    if (settingSelect == 0 || settingSelect == 1) {
        int dir = 0;

        if (jx < 100) dir = -1;
```

```

else if (jx > 1000) dir = 1;

if (dir != 0) {
    if (settingSelect == 0) {
        sensorDistanceCm = clampFloat(sensorDistanceCm + dir *
SENSOR_DISTANCE_STEP_CM,
MIN_SENSOR_DISTANCE_CM,
MAX_SENSOR_DISTANCE_CM);
        bufWriteFloat(EEPROM_DISTANCE_OFFSET, sensorDistanceCm);
    } else {
        speedLimitCms = clampFloat(speedLimitCms + dir *
SPEED_LIMIT_STEP_CMS,
MIN_SPEED_LIMIT_CMS,
MAX_SPEED_LIMIT_CMS);
        bufWriteFloat(EEPROM_SPEED_LIMIT_OFFSET, speedLimitCms);
    }
    delay(150);
}

if (cPressed && settingSelect == 2) {
    speedUnitSetting = 1 - speedUnitSetting;
    bufWriteInt(EEPROM_SPEED_UNIT_OFFSET, speedUnitSetting);
    if (buzzerSetting == 1) tone(BUZZER, 1200, 80);
    delay(150);
}

if (cPressed && settingSelect == 3) {
    ledSetting = !ledSetting;
    bufWriteInt(EEPROM_LED_OFFSET, ledSetting); // Write to RAM,
flush on B
    if (buzzerSetting == 1) tone(BUZZER, 1200, 80);

```

```

        delay(150);
    }

    if (cPressed && settingSelect == 4) {
        buzzerSetting = !buzzerSetting;
        bufWriteInt(EEPROM_BUZZER_OFFSET, buzzerSetting);
        if (buzzerSetting == 1) tone(BUZZER, 1200, 80);
        delay(150);
    }

    if (cPressed && settingSelect == 5) {
        // Reset counters; no need to physically erase records
        // because we only read up to saveNum
        saveNum = 0;
        saveIndex = 0;
        bufWriteInt(0, saveNum);
        bufWriteInt(4, saveIndex);
        eeprom_buffer_flush(); // Write to flash immediately so the
result is visible

        Serial.println("[EEPROM] Memory reset");
        if (buzzerSetting == 1) {
            tone(BUZZER, 600, 100); delay(120);
            tone(BUZZER, 800, 100);
        }
        delay(200);
    }

    display.setCursor(0, 0);
    display.setCursor(24, 0);
    display.println("-- SETTINGS --");

```

```
display.setCursor(0, 10);

display.setCursor(0, 10);
display.print(settingSelect == 0 ? ">" : " ");
display.print("DIST:");
display.print((int)sensorDistanceCm);
display.print("cm");

drawSlider(78, 10, 48, 7, sensorDistanceCm, MIN_SENSOR_DISTANCE_CM,
MAX_SENSOR_DISTANCE_CM);

display.setCursor(0, 19);
display.print(settingSelect == 1 ? ">" : " ");
display.print("LIM:");
if (speedUnitSetting == 1) {
    display.print(speedForDisplay(speedLimitCms), 1);
    display.print("kmh");
} else {
    display.print((int)speedLimitCms);
    display.print("cms");
}

drawSlider(78, 19, 48, 7, speedLimitCms, MIN_SPEED_LIMIT_CMS,
MAX_SPEED_LIMIT_CMS);

display.setCursor(0, 28);
display.print(settingSelect == 2 ? ">" : " ");
display.print("UNIT:");
display.print(speedUnitLabel());

display.setCursor(0, 37);
display.print(settingSelect == 3 ? ">" : " ");
display.print("LED:");
```

```
display.print(ledSetting == 1 ? "ON" : "OFF");

display.setCursor(0, 46);
display.print(settingSelect == 4 ? ">" : " ");
display.print("BUZZER:");
display.print(buzzerSetting == 1 ? "ON" : "OFF");

display.setCursor(0, 55);
display.print(settingSelect == 5 ? ">" : " ");
display.print("RESET MEMORY");
}
display.display();

lastBtnC = btnC;
lastBtnB = btnB;
lastBtnA = btnA;

if (jsw && (millis() - lastBeep > 300)) {
    if (buzzerSetting == 1) {
        tone(BUZZER, 1200, 80);
    }
    lastBeep = millis();
}
}
```